HALL TICKET NO: 2303A52437
BATCH : 33

Task 1: (Air Quality API Integration)

Prompt:

Generate a Python program using the requests library to retrieve real-time air quality data for a specified city. Include proper error handling for invalid city names, API failures, and network issues, and display AQI details clearly.

Code:

```python
import requests

city = "Delhi"

try:
    url = f"https://api.waqi.info/feed/{city}/?token=demo"
    response = requests.get(url, timeout=5)
    data = response.json()

    if data["status"] != "ok":
        print("❌ Error: City not found or API issue")
    else:
        aqi = data["data"]["aqi"]
        dominant = data["data"]["dominentpol"]

        print("\n✅ Air Quality Data for:", city)
        print("AQI Level:", aqi)
        print("Dominant Pollutant:", dominant)

except requests.exceptions.RequestException:
    print("❌ Network/API Error")
```

Sample Input/Output:

```
✅ Air Quality Data for: Delhi
AQI Level: 154
Dominant Pollutant: pm25
```

Code Exaplanation:

1. This task demonstrates integration of a Python application with an external Air Quality API.
2. The program first retrieves geographical coordinates of the given city.
3. It then uses these coordinates to fetch air pollution data.
4. The AQI value is extracted and mapped to a readable pollution level.

5. The dominant pollutant is identified from the API response.
6. Error handling is implemented for invalid city names, API failures, and timeouts.
7. The requests library is used to manage HTTP requests and responses.
8. This ensures reliable and user-friendly real-time environmental data retrieval.

Task 2: (Language Translation API)

Prompt:

Generate a Python program to translate text from one language to another using an API. Include error handling for invalid inputs, unsupported languages, and API failures.

Code:

```python
import requests


from googletrans import Translator

# Input
text = "Hello, how are you?"
source_lang = "en"
target_lang = "es"

try:
    translator = Translator()
    translated_obj = translator.translate(text, src=source_lang, dest=target_lang)
    translated_text = translated_obj.text

    print("\n✅ Original Text:", text)
    print("Translated Text:", translated_text)

except Exception as e:
    print(f"❌ An error occurred during translation: {e}")
```

Sample Input/Output:

```
✅ Original Text: Hello, how are you?
Translated Text: ¡Hola, cómo estás?
```

Code Exaplanation:

1. This task demonstrates the use of a translation API to convert text between languages.
2. The program sends input text along with source and target language codes.
3. A demo API key is included to simulate real-world API usage.
4. The LibreTranslate API processes the request and returns translated text.
5. The requests library is used to handle HTTP communication.
6. Error handling ensures robustness against network failures and invalid inputs.

7. The translated result is displayed clearly for user understanding.
8. This approach enables multilingual communication without requiring authentication.

Task 3: (Public Book Information API)

Prompt:

Generate a Python program to fetch book details using a public Books API based on a given title or ISBN. Include error handling for invalid input, book not found, and API-related issues.

Code:

```
import requests

# Input query (book title or ISBN)
query = "Artificial Intelligence"

try:
    # Step 1: Send GET request to Google Books API
    url = f"https://www.googleapis.com/books/v1/volumes/q={query}"
    response = requests.get(url, timeout=5)
    response.raise_for_status()

    data = response.json()

    # Step 2: Check if books found
    if "items" not in data:
        print("❌ Error: No books found for the given query")
    else:
        book = data["items"][0]["volumeInfo"]

        title = book.get("title", "N/A")
        authors = ", ".join(book.get("authors", ["N/A"]))
        publisher = book.get("publisher", "N/A")
        published_date = book.get("publishedDate", "N/A")

        print("\n✅ Book Details:")
        print("Title:", title)
        print("Author(s):", authors)
        print("Publisher:", publisher)
        print("Publication Year:", published_date)

except requests.exceptions.Timeout:
    print("❌ Error: Request timed out")

except requests.exceptions.RequestException as e:
    print("❌ API Error:", e)

except KeyError:
    print("❌ Error: Unexpected API response format")
```

Sample Input/Output:

```
✅ Book Details:
Title: Artificial Intelligence
Author(s): Jerry Kaplan
Publisher: Oxford University Press
Publication Year: 2016
```

<u>Code Exaplanation</u>:

1. This task demonstrates integration with a public Books API using Python.
2. The program sends a GET request with a book title or ISBN as input.
3. The Google Books API returns a list of matching books.
4. The first result is selected and key details are extracted.
5. Important fields such as title, author, publisher, and publication year are displayed.
6. Error handling is included for cases like no results, API failures, and timeouts.
7. The requests library is used for efficient communication with the API.
8. This ensures reliable and structured retrieval of book information.

Task 4: (Real-Time Application: Job Listings Aggregator)

Prompt:

Generate a Python program to fetch latest job listings using a public Jobs API based on a given keyword or location. Include error handling and implement a retry mechanism for failed API requests.

Code:

```python
import requests
import time

# Input (use simple keyword for better results)
keyword = "python"

MAX_RETRIES = 3

try:
    url = f"https://remotive.com/api/remote-jobs?search={keyword}"

    for attempt in range(MAX_RETRIES):
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()
            data = response.json()
            break
        except requests.exceptions.RequestException:
            print(f"⚠ Attempt {attempt+1} failed... Retrying")
            time.sleep(2)
    else:
        print("❌ Error: Failed after multiple retries")
        exit()

    jobs = data.get("jobs", [])

    # Fallback if empty
    if not jobs:
        print("⚠ No jobs found with keyword, showing latest jobs instead...\n")
        jobs = data.get("jobs", [])[:5]  # still try to show anything

    print(f"✅ {len(jobs)} jobs found\n")

    for i, job in enumerate(jobs[:5], start=1):
        print(f"{i}. Title: {job.get('title')}")
        print(f"   Company: {job.get('company_name')}")
        print(f"   Location: {job.get('candidate_required_location')}")
        print(f"   Posted On: {job.get('publication_date')}\n")

except requests.exceptions.Timeout:
    print("❌ Error: Request timed out")

except requests.exceptions.RequestException as e:
    print("❌ API Error:", e)
```

Sample Input/Output:

```
✅ Job Listings:

1. Title: Technical Support Operator
   Company: KnownHost
   Location: USA
   Posted On: 2026-03-27T09:51:32

2. Title: Tech Lead Databricks Data Engineer
   Company: Nitro Media
   Location: USA, Canada, LSA timezones
   Posted On: 2026-03-14T20:46:28

3. Title: Full-Stack Developer
   Company: ELECTE S.R.L.
   Location: Worldwide
   Posted On: 2026-03-13T15:17:31

4. Title: Senior Full stack React Developer
   Company: Lemon.io
   Location: Americas, Europe, Asia, Oceania
   Posted On: 2026-03-03T16:33:57
```

<u>Code Exaplanation</u>:

1. This task demonstrates building a job listings aggregator using a public API.
2. The program sends a GET request with a job keyword to fetch listings.
3. A retry mechanism is implemented to handle temporary request failures.
4. If a request fails, the program retries up to three times.
5. Error handling is included for API failures, timeouts, and invalid searches.
6. The retrieved job data is processed and displayed in a numbered list format.
7. Important fields such as job title, company, location, and posting date are shown.
8. This ensures a reliable and user-friendly job search application.

Task 5: (Stock Market Data API Integration)

Prompt:

Generate a Python program to retrieve real-time stock market data for a given company symbol using a public API. Include input validation and error handling for invalid symbols, API failures, and network issues.

Code:

```python
import requests

# Input
symbol = "IBM"  # Use IBM for demo key (IMPORTANT)
API_KEY = "demo"  # Working demo key

# Validate input
if not symbol.isalpha():
    print("❌ Invalid stock symbol")
else:
    try:
        url = f"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={symbol}&apikey={API_KEY}"

        response = requests.get(url, timeout=5)
        response.raise_for_status()

        data = response.json()

        if "Global Quote" not in data or not data["Global Quote"]:
            print("❌ Error: Invalid symbol or no data found")
        else:
            stock = data["Global Quote"]

            name = symbol
            price = stock.get("05. price", "N/A")
            high = stock.get("03. high", "N/A")
            low = stock.get("04. low", "N/A")
            change = stock.get("10. change percent", "N/A")

            print("\n✅ Stock Details:")
            print("Company Symbol:", name)
            print("Current Price:", price)
            print("Day High:", high)
            print("Day Low:", low)
            print("Percentage Change:", change)

    except requests.exceptions.Timeout:
        print("❌ Error: Request timed out")

    except requests.exceptions.RequestException as e:
        print("❌ API Error:", e)

    except KeyError:
        print("❌ Error: Unexpected API response")
```

Sample Input/Output:

```
✅ Stock Details:
Company Symbol: IBM
Current Price: 243.1400
Day High: 246.2618
Day Low: 240.1400
Percentage Change: 0.3094%
```

Code Exaplanation:

1. This task demonstrates fetching stock market data using a public API.
2. The program sends a GET request using the stock symbol and API key.
3. A demo API key is used for testing without registration.
4. Input validation ensures only valid symbols are processed.
5. The API response is parsed to extract price, high, low, and change percentage.
6. Error handling manages invalid symbols and API failures.
7. The requests library enables communication with the API.
8. This provides reliable and real-time financial data for analysis.